

```
Algorithm isFound(A, n, Val)
{
  if A[0] = Val then
    Return 0
  Return isFound(A+1, n-1, Val)
}

Algorithm FindMissing(A, n, current)
{
  if current > n+1 then
    return current
  if isFound(A, n, current) = 0 then
    return current
  return FindMissing(A, n, current+1)
}

Algorithm missingIntegerRecursive(A, n)
return FindMissing(A, n, 1)
}

Algorithm main()
{
  write "enter array size:"
  read n
  for i := 0 to n-1 do
  {
    write "enter element at index ", i, ":"
    read A[i]
  }
  Result := missingIntegerRecursive(A, n)
  write "the smallest missing positive integer is: ", Result
}
```

No Analysis

In function is found each recursive call decrements n by 1

$$\begin{aligned}\text{Recurrence is: } T(n) &= T(n-1) + c \\ &= T(n-2) + 2c \\ &= T(n-3) + 3c \\ &= T(0) + nc\end{aligned}$$

$$T(n) = O(n)$$

In find missing: For every value of current is found called and takes $O(n)$.

because is found call and recursive call

$$\text{recurrence relation} = T(n) = T(n-1) + n$$

$$T(n) = T(n-2) + T(n-1) + n$$

$$\begin{aligned}T(n) &= T(n-3) + T(n-2) + T(n-1) + n \\ &= \frac{n(n+1)}{2}\end{aligned}$$

$$T(n) = O(n^2)$$

$$\text{Best case} = O(n) \quad \text{Worst case} = O(n^2)$$

$$\text{Time complexity} = O(n^2)$$

↳ For overall algorithm

94/11